

COMPUTACIÓN CIENTÍFICA

Trabajo Práctico 1

Fecha límite: 23 de mayo de 2026 – 19:00 hs

BUENAS PRÁCTICAS

- Antes de entregar el trabajo, llenar el formulario y seguir las instrucciones allí presentes.
- Aplicar rigor matemático en las explicaciones. Redactar con letra clara.

A. ENTENDIENDO A LA MÁQUINA

1. En la bibliografía se suele definir al *épsilon de máquina* como el menor número positivo en la máquina tal que $1 + \varepsilon \neq 1$. Explique por qué dicho número merece una definición. En su explicación deberá integrar los siguientes conceptos:
 - (a) Ilustrar cómo se distribuyen los números de máquina (inventar máquinas imaginarias pequeñas). Realizar esto en base 2 y 10.
 - (b) Responder a la pregunta, ¿por qué $1 + \varepsilon$ y no (otro número) $+ \varepsilon$?
 - (c) Su relación con el error relativo.
 - (d) Si β es la base, β^{1-t} .
2. Escribir el pseudocódigo (puede guiarse por ejemplos de Burden o investigar en internet) de un algoritmo para calcular el épsilon de máquina en un sistema de aritmética de punto flotante con base β .
 - (a) Implementarlo en Python en forma de una función utilizando las buenas prácticas. ¿Cuál es el épsilon de máquina de Python? ¿Cómo se relaciona con **1.(d)**?
 - (b) Implementar una función que calcule el sucesor y el antecesor de un número. Probarla con los siguientes números: 1, 2 y 10. Agregue algún o algunos ejemplos de su autoría y relacione con lo hecho en **1.**. Luego calcular a mano los sucesores y antecesores y chequear si la función está dando valores correctos.

B. THE ART OF COMPUTING

Considerar el siguiente procedimiento para determinar el límite $\lim_{h \rightarrow 0} (e^h - 1)/h$ en una computadora. Sea

$$d_n = \text{fl} \left(\frac{e^{2^{-n}} - 1}{2^{-n}} \right) \quad \text{para } n = 0, 1, 2, \dots$$

y aceptemos como el límite de la máquina al primer valor que satisfaga $d_n = d_{n-1}$ (con $n \geq 1$).

1. Escribir y ejecutar una rutina en Python que implemente este procedimiento.
2. En una aritmética de punto flotante $\mathbb{R}(t, s)$, con redondeo por truncamiento (*chopping*) y utilizando la expansión de Taylor para e^h , ¿para qué valor de n se alcanzará la convergencia numérica, asumiendo que no ocurre desbordamiento a cero (*underflow*) de 2^{-n} ? Comparar este resultado con el experimento realizado en el punto (a) y exhibir conclusiones.
3. ¿En qué tipo de computadora (es decir, bajo qué condiciones sobre s y t) ocurrirá un error de *underflow* antes de que se alcance el límite?
4. Explicar qué limitación del cálculo numérico se está analizando en cada uno de los tres puntos anteriores.